

Lecture 8: Multiplicative Weights

Notes by Ola Svensson¹

These notes are based on [1] and [2].

1 The Multiplicative Weights Algorithm

This section is basically taken verbatim from the excellent lecture notes² by Anupam Gupta available here (together with a lot of other interesting information): <https://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15859-f11/www/>

Multiplicative weights is a retronym for the simple iterative rule that underlies modular, iterative algorithms for solving LPs and SDPs (semidefinite programs)³. The Multiplicative Weights Algorithm is known by different names in the various fields where it was (re)discovered. Check out the survey by Arora, Hazan and Kale [3]; our discussion will be based on their treatment. Due to its broad appeal, we will consider multiplicative weights in more generality than is needed for solving LPs and SDPs. In this lecture, we'll introduce some strategies for playing a prediction game.

1.1 Warmup: Prediction with Expert Advice

The following sequential game is played between an omniscient Adversary and an Aggregator who is advised by N experts. Special cases of this game include predicting if it will rain tomorrow, or if the stock market will go up or down.

Strategy 1 Prediction with Expert Advice

- 1: **for** $t = 1 \dots T$ **do**
 - 2: Each expert $i \in [N]$ advises either YES or NO
 - 3: Aggregator predicts either YES or NO
 - 4: Adversary, with knowledge of the expert advice and Aggregator's prediction, decides the YES/NO outcome.
 - 5: Aggregator observes the outcome and suffers if his prediction was incorrect.
 - 6: **end for**
-

Naturally, Aggregator wants to make as few mistakes as possible. Since the experts may be unhelpful and the outcomes may be capricious, Aggregator can hope only for a relative performance guarantee. In particular, Aggregator hopes to do as well as the best single expert in hindsight⁴. In order to do so, Aggregator must track which experts are helpful. We will consider a few tracking strategies. Almost every other aspect of the game - that advice is aggregated into a single value, that this value is binary, and even that the game is sequential - is not relevant; we will generalize or eliminate these aspects.

If there is a perfect expert, then an obvious strategy is to dismiss experts who are not perfect. With the remaining experts, take a majority vote. Whenever the Aggregator makes a mistake, at least half of the remaining experts are dismissed, so Aggregator makes at most $\log N$ mistakes⁵. We can use the same strategy even when there isn't a perfect expert, if we restart after every expert has been eliminated. If the best expert has made M mistakes by time T , then Aggregator has restarted at most $M + 1$ times, so it has made at most $(M + 1) \cdot \log N$ mistakes. This bound is rather poor since it depends multiplicatively on M .

¹**Disclaimer:** These notes were written as notes for the lecturer. They have not been peer-reviewed and may contain inconsistent notation, typos, and omit citations of relevant works.

²This section corresponds to Lecture 16 in the course, which was scribed by Shiva Kaul

³See Lecture 17 in the course linked earlier

⁴The excess number of mistakes is called (external) regret

⁵We will use $\log$ to denote the binary logarithm

1.2 Fewer Mistakes with Weighted Majority

We may obtain an additive mistake bound by softening our strategy: instead of dismissing experts who erred, discount their advice. This leads to the *Weighted Majority Algorithm* of Littlestone and Warmuth [4]. Assign each expert i a weight $w_i^{(1)}$ initialized to 1. Then for every t , predict YES/NO based on the weighted majority vote and halve the mistaken experts' weights after observing the outcome. The game strategy then becomes:

Strategy 2 Prediction with Weighted Majority

- 1: Initialize $\mathbf{w}^{(1)} = (w_1^{(1)}, \dots, w_N^{(1)})$ to be a vector of 1's
 - 2: **for** $t = 1 \dots T$ **do**
 - 3: Each expert $i \in [N]$ advises either YES or NO
 - 4: Aggregator predicts either YES or NO based on a weighted majority vote using $\mathbf{w}^{(t)}$
 - 5: Adversary, with knowledge of the expert advice and Aggregator's prediction, decides the YES/NO outcome.
 - 6: Aggregator observes the outcome and for every mistaken expert i , set $w_i^{(t+1)} \leftarrow w_i^{(t)}/2$
 - 7: **end for**
-

Claim 1 *For any sequence of outcomes, duration T , let M_{WM} be the number of mistakes that the Weighted Majority strategy makes, and M_i be the number of mistakes that expert i makes. Then*

$$M_{WM} \leq 2.41 \cdot (M_i + \log N)$$

Proof Let

$$\Phi^{(t)} = \sum_{i \in [N]} w_i^{(t)}$$

be a 'potential' function. Observe that:

- By definition, we have $\Phi^{(1)} = N$
- Also by definition, $\left(\frac{1}{2}\right)^{M_i} \leq \Phi^{(T+1)}$
- At any time τ when WM errs, at least half of the weight gets halved:

$$\Phi^{(\tau+1)} \leq \frac{3}{4} \Phi^{(\tau)}$$

This implies

$$\Phi^{(T+1)} \leq \left(\frac{3}{4}\right)^{M_{WM}} \Phi^{(1)}$$

Combining these facts yields

$$\left(\frac{1}{2}\right)^{M_i} \leq \left(\frac{3}{4}\right)^{M_{WM}} N$$

Taking the base-2 logarithm of both sides,

$$-M_i \leq \log N + \log \left(\frac{3}{4}\right) M_{WM}$$

so

$$M_{WM} \leq \underbrace{\frac{1}{\log(4/3)}}_{\approx 2.41} (M_i + \log N)$$

■

The leading constant is a consequence of our arbitrary choice to halve the weights. We can optimize ϵ in the update rule

$$w_i^{(t+1)} \leftarrow w_i^{(t)} / (1 + \epsilon)$$

then using the WM strategy we may achieve

$$M_{WM} \leq 2(1 + \epsilon)(M_i + O(\log N/\epsilon))$$

2 Changing the game: allowing for randomized strategies

In the exercises, you proved that there are instances for which weighted majority does twice as many mistakes as the best expert! This is undesirable. To overcome this limitation, we will allow the aggregator to play a random strategy instead of always making a deterministic prediction (that the adversary then uses to create bad instances). A side note is that this is often a general strategy: randomization is often very good to limit the effect of adversaries.

Allowing for randomized strategies leads to the following game with T days and N experts:

For $t = 1, \dots, T$:

1. Each expert $i \in [N]$ gives some advice.
2. Allocator picks some distribution $\vec{p}^{(t)} = (p_1^{(t)}, \dots, p_N^{(t)})$ over the experts.
3. Adversary, with knowledge of the expert advice and $\vec{p}^{(t)}$, determines a cost vector $\vec{m}^{(t)} = (m_1^{(t)}, \dots, m_N^{(t)}) \in [-1, 1]^N$.
4. Allocator observes the cost vector and suffers $\vec{p}^{(t)} \cdot \vec{m}^{(t)} = \sum_{i \in [N]} p_i^{(t)} m_i^{(t)}$.

Note that in the above game, we have abstracted away the given advice and in fact Step 1 is not important. At each day t , the goal is to find a distribution $\vec{p}^{(t)}$ over the experts so as to minimize the suffering (the cost). For each day t , the adversary decides that the cost of following the i 'th expert's advice is $m_i^{(t)}$. Here, we have generalized the cost to be anything in $[-1, 1]$ instead of only counting the number of mistakes. (A negative number means that it was profitable to follow that expert's advice.) As we play a randomized strategy, the expected cost incurred at day t is thus

$$\sum_{i \in [N]} \Pr[\text{aggregator follows expert } i \text{ at day } t] \cdot m_i^{(t)} = \sum_{i \in [N]} p_i^{(t)} m_i^{(t)} =: \vec{p}^{(t)} \cdot \vec{m}^{(t)}.$$

The ultimate goal is to design a strategy that does as well as the best expert. That is, we wish to find a strategy such that for every $i \in [N]$

$$\underbrace{\sum_{t=1}^T \vec{p}^{(t)} \cdot \vec{m}^{(t)}}_{\text{our loss}} \leq \underbrace{\sum_{t=1}^T \vec{m}_i^{(t)}}_{\text{loss of best expert}} + (\text{small error terms}).$$

The Hedge strategy that we explain in the next section accomplishes this.

Example 1 A natural example of the above situation is when you invest money. You have the option to choose between say 4 different funds provided by Postfinance, UBS, Credit Suisse, and Banque cantonale. The probability $\vec{p}^{(t)}$ vector that you choose at day t then corresponds to what fraction of your investment goes to the different funds. The goal is to update these fractions so as to make as much money as the best of the 4 options over the long run. Here the penalties (cost or profit) $\vec{m}^{(t)}$ that you observe reflects the performance of the different investments.

It is quite remarkable that you can do (almost) as good as the best possible option by doing rebalancing. *HOWEVER*, the caveat “almost” assumes that you live a very long life. For a small T , the guarantees get slightly worse.

3 The Hedge strategy

We now explain and analyze the Hedge strategy for the setting introduced in the last section. It is parameterized by the “learning parameter” $\epsilon > 0$:

- Initially, assign each expert i a weight $w_i^{(1)}$ of 1. (All experts are equally trustworthy in the beginning.)

At each time t :

- Pick the distribution $p_i^{(t)} = w_i^{(t)} / \Phi^{(t)}$ where $\Phi^{(t)} = \sum_{i \in [N]} w_i^{(t)}$.
- After observing the cost vector, set $w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\epsilon \cdot m_i^{(t)}}$.

The difference between Hedge and the so-called Multiplicative Weight Update method is the weight-update $w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\epsilon \cdot m_i^{(t)}}$ instead of $w_i^{(t+1)} = w_i^{(t)} \cdot (1 - \epsilon \cdot m_i^{(t)})$. Both methods have similar guarantees:

Theorem 2 Suppose $\epsilon \leq 1$ and for $t \in [T]$, $\vec{p}^{(t)}$ is picked by Hedge. Then for any expert i ,

$$\sum_{t=1}^T \vec{p}^{(t)} \cdot \vec{m}^{(t)} \leq \sum_{t=1}^T m_i^{(t)} + \frac{\ln N}{\epsilon} + \epsilon T.$$

Note that in the above theorem, i can be chosen to be the best expert and thus Hedge does as well as the best expert plus some small error terms.

Proof Similar to the analysis of the weighted majority strategy, the proof goes by analyzing the potential $\Phi^t = \sum_{i \in [N]} w_i^{(t)}$.

Lower bound on $\Phi^{(T+1)}$: We lower bound the final potential as a function of i ’s performance:

$$\Phi^{(T+1)} = \sum_{j \in [N]} w_j^{(T+1)} \geq w_i^{(T+1)} = \exp(-\epsilon \sum_{t=1}^T m_i^{(t)}),$$

where the last equality follows from that the initial weight of i was one and every day t his weight was updated by $e^{-\epsilon m_i^{(t)}}$.

Upper bound on $\Phi^{(T+1)}$: We upper bound the final potential in terms of the total cost Hedge suffered. By definition,

$$\Phi^{(t+1)} = \sum_{j \in [N]} w_j^{(t+1)} = \sum_{j \in [N]} w_j^{(t)} \cdot \exp(-\epsilon m_j^{(t)}).$$

The exponentiated term is in $[-1, 1]$. Since $e^x \leq 1 + x + x^2$ for $x \in [-1, 1]$ (do a Taylor expansion),

$$\begin{aligned}
\sum_{j \in [N]} w_j^{(t)} \cdot \exp(-\epsilon m_j^{(t)}) &\leq \sum_{j \in [N]} w_j^{(t)} \left((1 - \epsilon m_j^{(t)} + \epsilon^2 (m_j^{(t)})^2) \right) \\
&\leq \sum_{j \in [N]} w_j^{(t)} (1 - \epsilon m_j^{(t)} + \epsilon^2) \quad (\text{since } (m_j^{(t)})^2 \leq 1) \\
&= \sum_{j \in [N]} w_j^{(t)} (1 + \epsilon^2) - \sum_{j \in [N]} \epsilon w_j^{(t)} m_j^{(t)} \\
&= \Phi^{(t)} (1 + \epsilon^2) - \epsilon \sum_{j \in [N]} \Phi^{(t)} p_j^{(t)} m_j^{(t)} \quad (\text{since } \Phi^{(t)} p_j^{(t)} = w_j^{(t)}) \\
&= \Phi^{(t)} (1 + \epsilon^2 - \epsilon \vec{p}^{(t)} \vec{m}^{(t)}) \\
&\leq \Phi^{(t)} \cdot \exp(\epsilon^2 - \epsilon \vec{p}^{(t)} \vec{m}^{(t)}) \quad (\text{since } 1 + x \leq e^x).
\end{aligned}$$

We therefore have

$$\begin{aligned}
\Phi^{(T+1)} &\leq \Phi^{(T)} \cdot \exp(\epsilon^2 - \epsilon \vec{p}^{(T)} \vec{m}^{(T)}) \\
&\leq \Phi^{(T-1)} \cdot \exp(\epsilon^2 - \epsilon \vec{p}^{(T-1)} \vec{m}^{(T-1)}) \cdot \exp(\epsilon^2 - \epsilon \vec{p}^{(T)} \vec{m}^{(T)}) \\
&\quad \vdots \\
&\leq \Phi^{(1)} \cdot \exp\left(\epsilon^2 T - \epsilon \sum_{t=1}^T \vec{p}^{(t)} \vec{m}^{(t)}\right) \\
&= N \cdot \exp\left(\epsilon^2 T - \epsilon \sum_{t=1}^T \vec{p}^{(t)} \vec{m}^{(t)}\right),
\end{aligned}$$

where for the equality we used that $\Phi(1) = N$ since each expert was initialized with a weight of 1.

The above bounds give us

$$\exp(-\epsilon \sum_{t=1}^T m_i^{(t)}) \leq \Phi^{(T+1)} \leq N \cdot \exp\left(\epsilon^2 T - \epsilon \sum_{t=1}^T \vec{p}^{(t)} \vec{m}^{(t)}\right).$$

Taking (natural) logarithms,

$$-\epsilon \sum_{t=1}^T m_i^{(t)} \leq \ln(N) + \epsilon^2 T - \epsilon \sum_{t=1}^T \vec{p}^{(t)} \vec{m}^{(t)}$$

and the final result follows by dividing by ϵ and rearranging the terms. ■

Remark The above proof may seem messy with all the inequalities. However, it follows a standard “framework”: upper and lower bound the potential function. These Multiplicative Weight Update type of algorithms are most often analyzed using this potential argument. Once you know that cool argument, the analysis is almost automatic modulo some rewriting.

For the application of solving covering LPs, it will be convenient to consider the *average cost* incurred per day. We also generalize so that the cost vector can take values in $[-\rho, \rho]^N$ where ρ is called the “width”. The proof is the same by scaling and we get the following corollary:

Corollary 3 Suppose $\epsilon \leq 1$ and for $t \in [T]$, $\vec{p}^{(t)}$ is picked by Hedge and cost vectors satisfy $\vec{m}^{(t)} \in [-\rho, \rho]^N$. If $T \geq (4\rho^2 \ln N)/\epsilon^2$, then for any expert i :

$$\frac{1}{T} \sum_{t=1}^T \vec{p}^{(t)} \cdot \vec{m}^{(t)} \leq \frac{1}{T} \sum_{t=1}^T m_i^{(t)} + 2\epsilon.$$

References

- [1] David Leydier and Samuel Grütter: *Scribes of Lecture 8 in Topics in TCS 2014*.
<http://theory.epfl.ch/courses/topicstcs/Lecture8.pdf>
- [2] Romain Edelmann & Florian Tramèr: *Scribes of Lecture 10 in Topics in TCS 2014*.
<http://theory.epfl.ch/courses/topicstcs/Lecture10.pdf>
- [3] Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: a meta algorithm and applications. Technical report, Princeton University, 2005. 16
- [4] Nick Littlestone and Manfred K. Warmuth. The weighted majority algorithm. In FOCS, pages 256–261, 1989. 16.1.1